

Generics | `super`

01 | Was bedeutet das Schlüsselwort `super` bei Generics in Java?

- **Frage**

Erklären Sie den Begriff "super" bei Generics in Java.

- **Musterlösung**

Das Schlüsselwort `super` wird bei Generics verwendet, um eine Untergrenze für den Typ einer generischen Klasse oder Methode zu definieren. Es bedeutet, dass der Typ oder ein Obertyp des angegebenen Typs verwendet werden kann.

02 | Was ist der Unterschied zwischen `super` und `extends` bei Generics in Java?

- **Frage**

Erläutern Sie den Unterschied zwischen `super` und `extends` bei Generics in Java.

- **Musterlösung**

- `super`: Definiert eine Untergrenze für den Typ. Der Typ oder ein Obertyp des angegebenen Typs kann verwendet werden.
 - `extends`: Definiert eine Obergrenze für den Typ. Der Typ oder ein Untertyp des angegebenen Typs kann verwendet werden.
-

03 | Was ist der Unterschied zwischen `super` und `?` bei Generics in Java?

- **Frage**

Erläutern Sie den Unterschied zwischen `super` und `?` bei Generics in Java.

- **Musterlösung**

- `super`: Definiert eine Untergrenze für den Typ. Der Typ oder ein Obertyp des angegebenen Typs kann verwendet werden.
 - `?`: Ist ein Platzhalter für einen unbekannten Typ.
-

04 | Was ist der Unterschied zwischen `super` und `T` bei Generics in Java?

- **Frage**

Erläutern Sie den Unterschied zwischen `super` und `T` bei Generics in Java.

- **Musterlösung**

- `super`: Definiert eine Untergrenze für den Typ. Der Typ oder ein Obertyp des angegebenen Typs kann verwendet werden.
- `T`: Ist ein Typ-Parameter, der einen bestimmten Typ repräsentiert. Beispiel:

```
public <T> void processList(List<T> list) {  
    // erlaubt, da ein bestimmter Typ verwendet wird  
}
```

05 | Was ist der Unterschied zwischen `super` und `Object` bei Generics in Java?

- **Frage**

Erläutern Sie den Unterschied zwischen `super` und `Object` bei Generics in Java.

- **Musterlösung**

- `super`: Definiert eine Untergrenze für den Typ. Der Typ oder ein Obertyp des angegebenen Typs kann verwendet werden.
- `Object`: Ist die Oberklasse aller Klassen in Java. Beispiel:

```
public void processList(List<Object> list) {  
    // erlaubt, da nur Objekte vom Typ Object verwendet werden können  
}
```

06 | Was ist der Unterschied zwischen `super` und `Number` bei Generics in Java?

- **Frage**

Erläutern Sie den Unterschied zwischen `super` und `Number` bei Generics in Java.

- **Musterlösung**

- `super`: Definiert eine Untergrenze für den Typ. Der Typ oder ein Obertyp des angegebenen Typs kann verwendet werden.
- `Number`: Ist eine abstrakte Klasse, die die Oberklasse für numerische Typen wie `Integer`, `Double` usw. ist. Beispiel:

```
public void processNumbers(List<? super Number> list) {  
    // erlaubt, da Number oder ein Obertyp von Number verwendet wird  
}
```

07 | Was ist der Unterschied zwischen `super` und `Comparable` bei Generics in Java?

- **Frage** Erläutern Sie den Unterschied zwischen `super` und `Comparable` bei Generics in Java.

- **Musterlösung**

- `super`: Definiert eine Untergrenze für den Typ. Der Typ oder ein Obertyp des angegebenen Typs kann verwendet werden.
- `Comparable`: Ist eine Schnittstelle, die die Methode `compareTo(T o)` definiert, um Objekte zu vergleichen. Beispiel:

```
public void processComparables(List<? super Comparable> list) {  
    // erlaubt, da Comparable oder ein Obertyp von Comparable verwendet wird  
}
```

08 | Was ist der Unterschied zwischen `super` und `List` bei Generics in Java?

- **Frage**

Erläutern Sie den Unterschied zwischen `super` und `List` bei Generics in Java.

- **Musterlösung**

- `super`: Definiert eine Untergrenze für den Typ. Der Typ oder ein Obertyp des angegebenen Typs kann verwendet werden.
- `List`: Ist eine Schnittstelle, die eine geordnete Sammlung von Elementen repräsentiert. Beispiel:

```
public void processLists(List<? super List> list) {  
    // erlaubt, da List oder ein Obertyp von List verwendet wird  
}
```

09 | Was ist der Unterschied zwischen `super` und `Collection` bei Generics in Java?

- **Frage** Erläutern Sie den Unterschied zwischen `super` und `Collection` bei Generics in Java.

- **Musterlösung**

- `super`: Definiert eine Untergrenze für den Typ. Der Typ oder ein Obertyp des angegebenen Typs kann verwendet werden.
- `Collection`: Ist eine Schnittstelle, die eine Sammlung von Elementen repräsentiert. Beispiel:

```
public void processCollections(List<? super Collection> list) {  
    // erlaubt, da Collection oder ein Obertyp von Collection verwendet wird  
}
```

10 | Was ist der Unterschied zwischen `super` und `Iterable` bei Generics in Java?

- **Frage**

Erläutern Sie den Unterschied zwischen `super` und `Iterable` bei Generics in Java.

- **Musterlösung**

- `super`: Definiert eine Untergrenze für den Typ. Der Typ oder ein Obertyp des angegebenen Typs kann verwendet werden.
- `Iterable`: Ist eine Schnittstelle, die die Methode `iterator()` definiert, um über eine Sammlung von Elementen zu iterieren. Beispiel:

```
public void processIterables(List<? super Iterable> list) {  
    // erlaubt, da Iterable oder ein Obertyp von Iterable verwendet wird  
}
```

11 | Was ist der Unterschied zwischen `super` und `Map` bei Generics in Java?

- **Frage** Erläutern Sie den Unterschied zwischen `super` und `Map` bei Generics in Java.
- **Musterlösung**
 - `super`: Definiert eine Untergrenze für den Typ. Der Typ oder ein Obertyp des angegebenen Typs kann verwendet werden.
 - `Map`: Ist eine Schnittstelle, die eine Sammlung von Schlüssel-Wert-Paaren repräsentiert. Beispiel:

```
public void processMaps(List<? super Map> list) {  
    // erlaubt, da Map oder ein Obertyp von Map verwendet wird  
}
```

12 | Was ist der Unterschied zwischen `super` und `Set` bei Generics in Java?

- **Frage**

Erläutern Sie den Unterschied zwischen `super` und `Set` bei Generics in Java.
- **Musterlösung**
 - `super`: Definiert eine Untergrenze für den Typ. Der Typ oder ein Obertyp des angegebenen Typs kann verwendet werden.
 - `Set`: Ist eine Schnittstelle, die eine Sammlung von eindeutigen Elementen repräsentiert. Beispiel:

```
public void processSets(List<? super Set> list) {  
    // erlaubt, da Set oder ein Obertyp von Set verwendet wird  
}
```

13 | Was ist der Unterschied zwischen `super` und `Queue` bei Generics in Java?

- **Frage**

Erläutern Sie den Unterschied zwischen `super` und `Queue` bei Generics in Java.
- **Musterlösung**
 - `super`: Definiert eine Untergrenze für den Typ. Der Typ oder ein Obertyp des angegebenen Typs kann verwendet werden.
 - `Queue`: Ist eine Schnittstelle, die eine Warteschlange von Elementen repräsentiert. Beispiel:

```
public void processQueues(List<? super Queue> list) {  
    // erlaubt, da Queue oder ein Obertyp von Queue verwendet wird  
}
```

14 | Was ist der Unterschied zwischen `super` und `Deque` bei Generics in Java?

- **Frage**

Erläutern Sie den Unterschied zwischen `super` und `Deque` bei Generics in Java.

- **Musterlösung**

- `super`: Definiert eine Untergrenze für den Typ. Der Typ oder ein Obertyp des angegebenen Typs kann verwendet werden.
- `Deque`: Ist eine Schnittstelle, die eine doppelseitige Warteschlange von Elementen repräsentiert. Beispiel:

```
public void processDeques(List<? super Deque> list) {  
    // erlaubt, da Deque oder ein Obertyp von Deque verwendet wird  
}
```

15 | Was ist der Unterschied zwischen `super` und `Stack` bei Generics in Java?

- **Frage**

Erläutern Sie den Unterschied zwischen `super` und `Stack` bei Generics in Java.

- **Musterlösung**

- `super`: Definiert eine Untergrenze für den Typ. Der Typ oder ein Obertyp des angegebenen Typs kann verwendet werden.
- `Stack`: Ist eine Klasse, die eine LIFO (Last-In-First-Out)-Struktur repräsentiert. Beispiel:

```
public void processStacks(List<? super Stack> list) {  
    // erlaubt, da Stack oder ein Obertyp von Stack verwendet wird  
}
```